Basic SQL Queries

Data-types based SQL Queries: int, number, integer, smallint, float, decimal

- create table e1(no int, grade float);
- create table e1(no integer, grade float);
- create table e1(no smallint, grade float);
- create table e1(no smallint, grade decimal(1,2));
- create table e1(no smallint, grade numeric(1,2));

Data-types based SQL Queries: char, varchar, clob, raw, bit, blob

- create table e1(eno int, name varchar(5));
- create table e1(eno int, name char(5));
- create table e1(eno int, name varchar(5), section char);
- create table e1(eno int, name varchar(5), section char, details clob);
- create table e1(no int, bt char(1) check (bt='1' or bt='0’));
- The RAW datatype is used for binary data or byte strings that are not to be interpreted by Oracle, for example, to store graphics character sequences:
  - create table e1(no int, ig raw(10));

Data-types based SQL Queries: date, timestamp, time interval

- create table e1(no int, d date);
- insert into e1 values(10,'22/APR/2011’);
- create table e1(no int, d Date, t timestamp);
- insert into e1 values(10,'10-APR-2011','10-APR-2011 10:30:00.123’);
- insert into e1 values(20,'10-APR-2011',current_timestamp);
- create table e1(no int, d date);
- insert into e1 values(10,'10-APR-2011’);
- select (d- interval '1' day) from e1;

Specifying Constraints in SQL statements

- NOT NULL
- Default
- Check
- Primary key
- Foreign key

Constraints in Create statement

- create table e2(no int primary key, name char(10) not null, age int check(age>20 and age<60));
- create table e3(no int primary key, name char(10) not null, age int default 20 check(age>20 and age<60));

On delete set null, on delete set default, on update cascade

- create table e1(ssn int primary key, name varchar(10));
- insert into e1 values (10, 'abc’);
- create table d1 (dno int primary key, mno int not null, mssn int default 1, foreign key(mssn) references e1(ssn));
- create table d2 (dno int primary key, mno int not null, mssn int default 1, foreign key(mssn) references e1(ssn) on delete set null);
- select * from e1;
- delete from e1 where ssn=10;
- select * from d2;
- DNO MNO MSSN
- ---------- ---------- ----------
- 1 1
- 2 2 20

Aggregate functions: max, min, avg, sum, count

- select max(sal) from e5;
- select min(sal) from e5;
- select avg(sal) from e5;
- select sum(sal) from e5;
- select count(eid) as cnts from e5;

Select with where conditions

- select eid from e5 where sal>20000;
- select eid from e5 where sal>20000 and sal<50000;
- select eid from e5 where sal<30000 or sal=30000;

Group by order by

- create table e5(eid int, sal float);
- insert into e5 values(&eid,&sal);
- select count(sal) from e5 group by sal order by sal;
- UPDATE t3 SET SAL = SAL *1.1;

Group by and order by

GROUP BY	ORDER BY
Group by statement is used to group the rows that have the same value.	Whereas Order by statement sort the result-set either in ascending or descending order.
It may be allowed in CREATE VIEW statement.	While it does not use in CREATE VIEW statement.
In select statements, it is always used before the order by keyword.	While in the select statement, it is always used after the group by keyword.
An attribute cannot be in the group by a statement under the aggregate function.	Whereas in order by statement, the attribute can be under aggregate function.
In group by clause, the tuples are grouped based on the similarity between the attribute values of tuples.	Whereas in order by clause, the result set is sorted based on ascending or descending order.
Group by controls the presentation of tuples(rows).	While order by clause controls the presentation of columns.

The SELECT-FROM-WHERE Structure of Basic SQL Queries

Q0: Retrieve the birthdate and address of the employees whose name is 'John B. Smith'.

QO: SELECT BDATE, ADDRESS FROM EMPLOYEE WHERE FNAME='John' AND MINIT='B' AND LNAME='Smith';

QUERY 1: Retrieve the name and address of all employees who work for the 'Research' department.

- create table emplyee (ssn int primary key, name varchar(20), dno int);
- insert into emplyee values(&ssn,'&name',&dno);
- create table department (dnumber int, dname varchar(20), address varchar(20));
- insert into department values(&dnumber,'&dname','&address’);
- select name,address from emplyee,department where dname='research' and dno=dnumber;

Query 2. For every project located in ‘Stafford’, list the project number, the controlling department number, and the department manager’s last name, address, and birth date.

- **select-project-join** query
- create table employee(ssn int primary key, name varchar(20), dno int not null);
- insert into employee values(&ssn,'&name',&dno);
- create table department (dnumber int, dname varchar(20), address varchar(20));
- insert into department values(&dnumber,'&dname','&address’);
- create table project1(pnumber int primary key, plocation varchar(20), mgrssn int not null);
- insert into project1 values(&pnumber,'&plocation',&mgrssn);
- select name,address, pnumber from employee,project1,department where dno=dnumber and mgrssn=ssn and plocation='stanford'

Ambiguous Attribute Names, Aliasing, Renaming, and Tuple Variables

- **Q1A: SELECT FNAME, EMPLOYEE.NAME, ADDRESS FROM EMPLOYEE,DEPARTMENT WHERE DEPARTMENT.NAME='Research' AND DEPARTMENT.DNUMSER=EMPLOYEE.DNUMSER;**

More queries: Select with where conditions combining two tables

QUERY 1: Retrieve the name and address of all employees who work for the 'Research' department.

Ql: SELECT FNAME,LNAME,ADDRESS FROM EMPLOYEE,DEPARTMENT WHERE DNAME='Research' AND DNUMBER=DNO;

QUERY 2: For every project located in 'Stafford', list the project number, the controlling department number, and the department manager's last name, address, and birthdate.

Q2: SELECT PNUMBER, DNUM, LNAME, ADDRESS, BDATE FROM PROJECT, DEPARTMENT, EMPLOYEE WHERE DNUM=DNUMBER AND MGRSSN=SSN AND PLOCATION='Stafford';

Ambiguous Attribute Names, Aliasing, and Tuple Variables

- **Q1A: SELECT FNAME, EMPLOYEE.NAME, ADDRESS FROM EMPLOYEE,DEPARTMENT WHERE DEPARTMENT.NAME='Research' AND DEPARTMENT.DNUMSER=EMPLOYEE.DNUMSER;**
- SELECT FROM WHERE E.FNAME, E.NAME, E.ADDRESS EMPLOYEE E, DEPARTMENT D D.NAME='Research' AND D.DNUMBER=E.DNUMBER;

QUERY 8 For each employee, retrieve the employee's first and last name and the first and last name of his or her immediate supervisor.

Q8: SELECT E.FNAME, E.LNAME, S.FNAME, S.LNAME FROM EMPLOYEE AS E, EMPLOYEE AS S WHERE E.SUPERSSN=S.SSN;

Unspecified WHERE Clause and Use of the Asterisk

Select all EMPLOYEE SSNS **(Q9),** and

all combinations of EMPLOYEE SSN and DEPARTMENT DNAME **(Q10)** in the database.

Q9: SELECT SSN FROM EMPLOYEE

QlO: SELECT SSN, DNAME FROM EMPLOYEE, DEPARTMENT;

QIC:

SELECT * FROM EMPLOYEE WHERE DNO=5;

QID: SELECT * FROM EMPLOYEE, DEPARTMENT WHERE DNAME='Research' AND DNO=DNUMBER

Tables as Sets in SQL

QUERY 11 Retrieve the salary of every employee (Qll) and all distinct salary values (QllA).

Qll: SELECT ALL SALARY FROM EMPLOYEE;

QIIA: SELECT DISTINCT SALARY FROM EMPLOYEE;

SQL has directly incorporated some of the set operations of relational algebra. There are set union (UNION), set difference (EXCEPT), and set intersection (INTERSECT) operations.

Tables as Sets in SQL

QUERY 4 Make a list of all project numbers for projects that involve an employee whose last name is 'Smith', either as a worker or as a manager of the department that controls the project.

Q4: (SELECT DISTINCT PNUMBER FROM PROJECT, DEPARTMENT, EMPLOYEE WHERE DNUM=DNUMBER AND MGRSSN=SSN AND LNAME='Smith') UNION (SELECT DISTINCT PNUMBER FROM PROJECT, WORKS_ON, EMPLOYEE WHERE PNUMBER=PNO AND ESSN=SSN AND LNAME='Smith');

Substring Pattern Matching and Arithmetic Operators

Retrieve all employees whose address is in Houston, Texas.

Q12: SELECT FNAME, LNAME FROM EMPLOYEE WHERE ADDRESS LIKE '%Houston,TX%';

To retrieve all employees who were born during the 1950s, we can use Query 12A. Here, '5' must be the third character of the string (according to our format for date), so we use the value '__ 5 ', with each underscore serving as a placeholder for an arbitrary character.

The IN operator checks if a value exists within a list or subquery result, while EXISTS checks if a subquery returns any rows, useful for existence checks, especially with large datase

Substring Pattern Matching and Arithmetic Operators

QUERY 12A Find all employees who were born during the 1950s.

Q12A: SELECT FROM FNAME, LNAME WHERE EMPLOYEE BDATE LIKE '_ _ 5_ _ _ _ _ _ _';

In and Exists operator

In Operator

The `IN` operator checks if a value exists within a list or subquery result, while `EXISTS` checks if a subquery
returns any rows, useful for existence checks, especially with large datasets

```
SELECT column_name(s) FROM table_name WHERE column_name IN
(value1, value2, ...);
```

EXISTS Operator:

- **Purpose:** Determines if a subquery returns any rows, used for existence checks.

- `SELECT column_name(s) FROM table_name WHERE EXISTS (SELECT * FROM another_table WHERE condition);`

Where and Having clause

- The SQL HAVING clause is used in combination with the GROUP BY to filter the results by conditions that involve aggregate functions. While the WHERE clause filters individual rows, the HAVING filters groups of rows based on aggregate values.

- **SELECT** column**1**, **COUNT(**column**2)** **FROM** table_name **GROUP BY** column**1** **HAVING** **COUNT(**column**2)** > **10**;

Substring Pattern Matching and Arithmetic Operators

QUERY 13 Show the resulting salaries if every employee working on the 'ProductX' project is given a 10 percent raise.

Q13: SELECT FNAME, LNAME, 1.1*SALARY AS INCREASED_SAL FROM EMPLOYEE, WORKS_ON, PROJECT

WHERE SSN=ESSN AND PNO=PNUMBER AND PNAME='ProductX';

For string data types, the concatenate operator I I can be used in a query to append two string values. For date, time, timestamp, and interval data types, operators include incrementing (+) or decrementing (-) a date, time, or timestamp by an interval.

Ordering of Query Results

SQL allows the user to order the tuples in the result of a query by the values of one or more attributes, using the ORDER BY clause.

QUERY 15 Retrieve a list of employees and the projects they are working on, ordered by department and, within each department, ordered alphabetically by last name, first name.

Q15: SELECT DNAME, LNAME, FNAME, PNAME FROM DEPARTMENT, EMPLOYEE, WORKS_ON, PROJECT WHERE DNUMBER=DNO AND SSN=ESSN AND PNO=PNUMBER ORDER BY DNAME, LNAME, FNAME;

Q16: ORDER BY DNAME DESC, LNAME ASC, FNAME ASC

INSERT, DELETE, AND UPDATE STATEMENTS IN SQL

INSERT is used to add a single tuple to a relation. We must specify the relation name and a list of values for the tuple.

>>>INSERT INTO EMPLOYEE VALUES ('Richard', 'K', 'Marini', '653298653', '1962-12-30', '98 Oak Forest, Katy, TX', 'M', 37000, '987654321', 4)

>>>INSERT INTO EMPLOYEE (FNAME, LNAME, DNO, SSN) VALUES('Richard', 'Marini', 4, '653298653');

INSERT INTO EMPLOYEE (FNAME, LNAME, DNO) VALUES ('Robert', 'Hatcher', 5);

(* It is rejected if not null checking is provided by dbms [ssn]*)

Delete command

The DELETE command removes tuples from a relation. It includes a WHERE clause, similar to that used in an SQL query, to select the tuples to be deleted.

>>>DELETE FROM EMPLOYEE WHERE LNAME='Brown';

>>>DELETE FROM EMPLOYEE

Update Command

The UPDATE command is used to modify attribute values of one or more selected tuples. An additional SET clause in the UPDATE command specifies the attributes to be modified and their new values.

An example is to give all employees in the 'Research' department a 10 percent raise in salary

UPDATE EMPLOYEE SET SALARY = SALARY *1.1 WHERE DNO IN (SELECT DNUMBER FROM DEPARTMENT WHERE DNAME='Research')

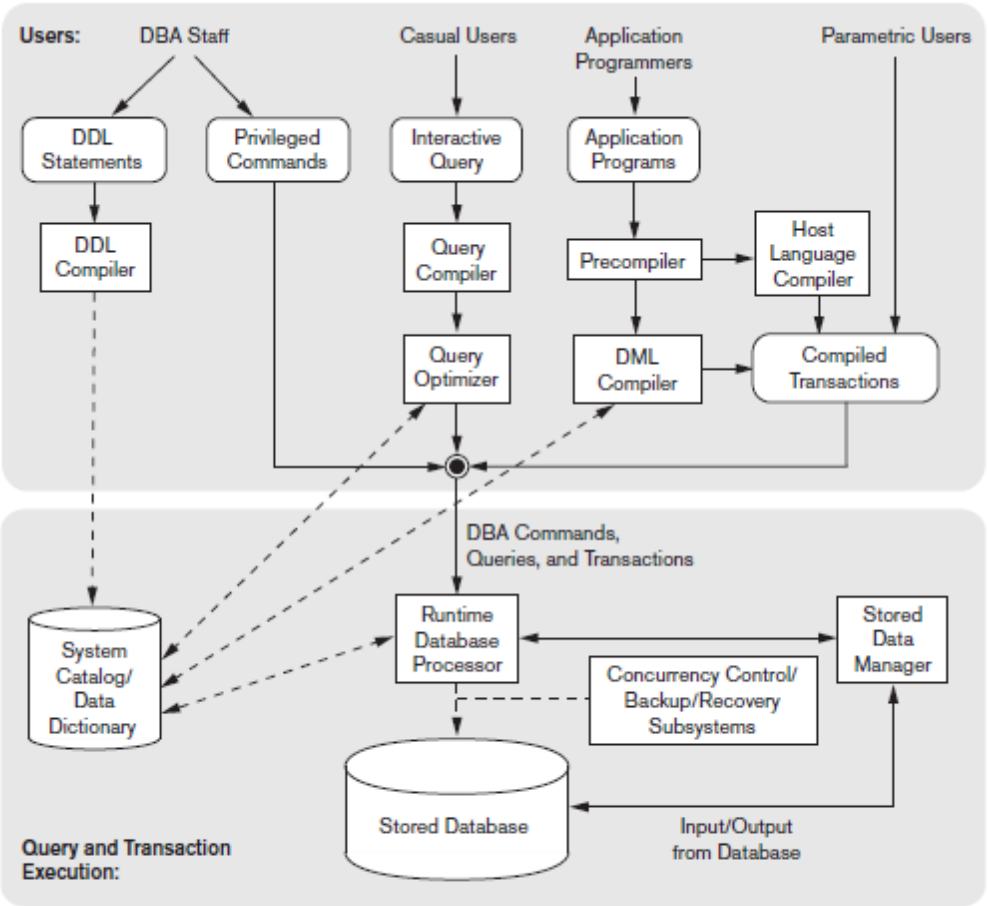

Figure 2.3
Component modules of a DBMS and their interactions.

Explanation

- Top of the diagram, DBA staff, casual users who work with interactive interfaces to formulate queries, application programmers who create programs using some host programming languages, and parametric users who do data entry work by supplying parameters to predefined transactions.
- The DBA staff works on defining the database and tuning it by making changes to its definition using the DDL and other privileged commands.
- The DDL compiler processes schema definitions, specified in the DDL, and stores descriptions of the schemas (meta-data) in the DBMS catalog.
- The catalog includes information such as the names and sizes of files, names and data types of data items, storage details of each file, mapping information among schemas, and constraints.
- Casual users and persons with occasional need for information from the database interact using the **interactive query** interface in Figure 2.3.
- These queries are parsed and validated for correctness of the query syntax, the names of files and data elements, and so on by a **query compiler** that compiles them into an internal form.
- .This internal query is subjected to query optimization **query optimizer** is concerned with the rearrangement and possible reordering of operations, elimination of redundancies, and use of efficient search algorithms during execution.
- It consults the system catalog for statistical and other physical information about the stored data and generates executable code that performs the necessary operations for the query and makes calls on the runtime processor.

- Application programmers write programs in host languages such as Java, C, or C++ that are submitted to a precompiler.
- The **precompiler** extracts DML commands from an application program written in a host programming language.
- These commands are sent to the DML compiler for compilation into object code for database access.
- The rest of the program is sent to the host language compiler.
- The object codes for the DML commands and the rest of the program are linked, forming a canned transaction whose executable code includes calls to the runtime database processor.
- the **runtime database processor** executes (1) the privileged commands, (2) the executable query plans, and (3) the canned transactions with runtime parameters.
- It works with the **system catalog** and may update it with statistics. It also works with the **stored data manager**, which in turn uses basic operating system services for carrying out low-level input/output (read/write) operations between the disk and main memory. The runtime database processor handles other aspects of data transfer, such as management of buffers in the main memory.

Lab experiment 4

- set linesize 200;

- EMPNO ENAME JOB HIREDATE MGR SAL COMM DNO
- --- --- --- --- --- --- --- ---
- 101 aa clerk 22-APR-22 10 50000 1
- 102 bb clerk 20-APR-21 10 50000 1
- 103 cc salesman 10-MAY-20 10 40000 1
- 104 dd Manager 10-MAY-20 9 80000 2
- 10 ee Manager 10-MAY-20 9 80000 2
- 105 ff salesman 10-MAY-20 10 40000 1
- 9 gg president 10-MAY-20 90000 1

- 7 rows selected.

1. **Display all the details of all Managers**

- select * from emp where job='Manager’;

2. **List the emps who joined before 2000.**

- select * from emp where hiredate < ’01-Jan-2021′ ;

3. List the Empno, Ename, Sal, Daily Sal of all Employees in the ASC order of AnnSal.

SELECT Empno, Ename, sal FROM Emp ORDER BY Sal*12

SELECT Empno, Ename, sal*12 FROM Emp ORDER BY Sal*12 ;

SELECT Empno, Ename, sal, Sal/30 DailySal FROM Emp ORDER BY Sal*12 ;

4. Display the empno , ename, job, hiredate, exp of all Mgrs

select empno, ename, sal, months_between(sysdate,hiredate)/12 Exp from emp where job='Manager’;

5. List the empno, ename, sal, exp of all emps working for Mgr 7839.

select empno, ename, sal, months_between(sysdate,hiredate)/12 Exp from emp B where Mgr=9;

6.Display the details of the emps whose Comm. Is more than their sal.

select * from emp where comm > sal ;

7. List the emps in the asc order of Designations

select * from emp order by job ;

8. **List the emps along with their exp and daily sal is more than Rs.100**

- Select emp.*, months_between(sysdate,hiredate)/12 Exp from emp where sal/30 > 100 ;

9. **List the emps who are either ‘CLERK’ or ‘ANALYST’ in the desc order**

- Select * from emp where job in (‘CLERK’,‘ANALYST’) order by job desc ;
- Select * from emp where job='clerk' or job='salesman' order by job desc ;

10. **List the emps who joined on 1-May-81,31-Dec-81, 17-Dec-81, 19-Jan-80 in asc order of seniority.**

- Select ename, hiredate, months_between(sysdate, hiredate)/12 EXP from emp where hiredate like ’10-MAY-2020′ or hiredate like ’10-MAY-2020′or hiredate like ’22-APR-2022′ or hiredate like ’22-APR-2021’ order by hiredate desc ;

11. List the emps who are working for the deptno 10 or 20

Select * from emp where dno in (1,2) ;

12. List the emps who are joined in the year 2020

Select * from emp where hiredate like '%20' ;

13. List the emps who are joined in the month of May 2020

Select * from emp where hiredate like '%MAY-20' ;

14. List the emps whose annul sal ranging from 45000 and 60000

Select * from emp where sal*12 between 45000 and 60000 ;

15. List the enames those are starting with 's' and with fire characters

Select * from emp where ename like 'S____' ;

16. **List the emps those are having four chars and third char must be 'r'**

Select * from emp where ename like '__R_' ;

17. List the 5 character names starting with ‘s’ and ending with ‘h’

- Select * from emp where ename like ‘S___H’ ;
- Select * from emp where ename like ‘S___H’ ;

18. List the emps who joined in January

- Select * from emp where hiredate like ‘%JAN%’ ;

19. List the emps who joined in the month of which second character is ‘a’

- Select * from emp where hiredate like ‘__-_A%’ ;

20. List the emps whose sal is 4 digit number ending with zero

- Select * from emp where sal like ‘___0’ ;

21. List the emps whose names having a character set ‘ll’ together

- Select * from emp where ename like ‘%LL%’ ;

22. List the emps who does not belong to deptno 20

- Select * from emp where deptno <> 20 ;

23. List all the emps except ‘president’ & ‘Mgr’ in asc order of salaries

- Select * from emp where job not in (‘PRESIDENT’,’MANAGER’)
- Select * from emp where job != 'president' order by sal;

24. List the emps who joined in before or after 2020

- Select * from emp where hiredate not like '%20’ ;

25. List the emps whose empno not starting with digit 78

- Select * from emp where empno not like ‘78%’ ;

26. List the emps who are working under ‘Mgr’

- Select * from emp where mgr in (select empno from emp where job ='Manager’);

27. List the emps who joined in any year but not belongs to the month of March

- Select * from emp where hiredate not like ‘%MAR%’ ;

28. **List the total information of emp table along with dname and loc of all the emps working under ‘Accounting’ & ‘Research’ in the asc deptno**

- select emp.*,dname from emp,dept where dname in ('d1','d2') and emp.dno=dept.dnum order by emp.dno;

29. List the details of the emps whose salaries more than the employee BLAKE

- select A.* from emp A, emp B where A.sal > B.sal and B.ename = ‘BLAKE’
- OR
- select * from emp where sal > (select sal from emp where ename = ‘BLAKE’)

30. List the details of the emps whose job is same as ALLEN.

- select * from emp where job = (select job from emp where ename = ‘ALLEN’)

31. List the emps who are senior to King

- select * from emp where hiredate < (select hiredate from emp where ename = ‘KING’)

32. Display the name of the employee who is drawing maximum salary

- Select ename from emp where sal in(select max(sal) from emp);

33. Display top 5 rows from a table

- Select * from emp where rownum<=5;

34. List the names,job,sal , dname deptwise

- Select e.ename,e.job, e.sal,d.dname from emp e, dept d where e.deptno =d.deptno order by e.deptno;